ALPE FORMACIÓN · SANTANDER

Edición 2026 · PHP 8.x

CURSO DE PHP · CLASE 38

Proyecto final y despliegue

- La academia completa
- Instalar el proyecto desde cero
- Buenas prácticas en Laravel
- Revisión de código
- Preparar la aplicación para producción
- Dónde publicar una aplicación Laravel
- Despliegue paso a paso en un servidor
- Actualizar sin romper nada
- El proyecto final



Ramón Abramo

+25 años de experiencia docente · Santander, Cantabria

ALPE

FORMACIÓN PROFESIONAL

🧭 Contenido de la clase

Llegamos a la última clase. Empezamos el curso sin saber qué era una variable, y hoy tenemos una aplicación con rutas, vistas, base de datos, relaciones, formularios, usuarios con roles, una API con tokens, pruebas automáticas, colas, correos y tareas programadas. No está nada mal, ¿eh? 😄

Hoy hacemos tres cosas: **juntar** la academia completa en una carpeta, aprender a **revisar** el código como se hace en una empresa y, sobre todo, **publicarla en Internet**, que una web que solo funciona en vuestro ordenador es media web. Y al final os propongo vuestro **proyecto final**.

•
👌 Cómo vamos a trabajar



1
🚀 La academia completa



2
👌 Instalar el proyecto desde cero



3
📘 Buenas prácticas en Laravel



4
🔍 Revisión de código



5
⚠️ Preparar la aplicación para producción



6
🤔 Dónde publicar una aplicación Laravel



7
📘 Despliegue paso a paso en un servidor



8
😮 Actualizar sin romper nada



9
🚀 El proyecto final



10
🚀 Ejercicios



11
👋 Lo que tenéis que llevaros del curso





CLASE 38

👌 Cómo vamos a trabajar

Esta clase es de proyecto, así que en lugar de `ejemplos/` tiene dos carpetas:

Esquema

```
proyecto/      ← la academia COMPLETA: todos los ficheros nuestros de las clases 27 a 37
despliegue/    ← los ficheros de configuración del servidor
   deploy.sh                        ← el script para actualizar la aplicación
   nginx-academia.conf              ← la configuración del servidor web
   supervisor-academia-worker.conf ← mantiene vivo el worker de la cola
   crontab.txt                      ← la línea del planificador de tareas
```



CLASE 38 · PUNTO 1

🚀 La academia completa

Esto es lo que hace nuestra aplicación y en qué clase lo construimos:

Funcionalidad	Dónde está	Clase
Configuración propia y comandos de Artisan	`config/academia.php`, `routes/console.php`	27 y 28
Portada, contacto y listado de cursos con buscador	`PaginaController`, `CursoController`	28
Layout, menú y componentes	`resources/views/layouts`, `components`	29
Tablas, datos de prueba y consultas	`database/`	30
Modelos con relaciones, *scopes*, *accessors* y paginación	`app/Models`	31
CRUD de cursos con imagen y validación en español	`CursoRequest`, `lang/es`	32
Login, registro, roles, *gates* y *policies*	`Auth/`, `Rol`, `CursoPolicy`	33
Servicios, contratos, fachada, middleware y *view composer*	`app/Servicios`, `AcademiaServiceProvider`	34
API REST versionada con tokens	`Api/V1`, `Resources`, `routes/api.php`	35
24 pruebas automáticas	`tests/`	36
Matrícula con evento, correos en cola, notificaciones, caché y tareas programadas	`Events`, `Listeners`, `Mail`, `Notifications`, `Jobs`	37

Además, para esta última versión he pasado **Pint** por todo el código (lo vemos en el punto 4).



💡 He dejado también las rutas de práctica de las primeras clases (`/hola` , `/blade` , `/edad/{edad}` ...). En vuestro proyecto final, borrad todo lo que no forme parte de la aplicación.



CLASE 38 · PUNTO 2

👌 Instalar el proyecto desde cero

Si os habéis perdido alguna clase o queréis empezar limpios, así se monta la academia completa:

terminal — php-paso-a-paso/clase38 — Visual Studio Code

terminal

```
composer create-project laravel/laravel academia-laravel
```

terminal — php-paso-a-paso/clase38 — Visual Studio Code

terminal

```
cd academia-laravel
composer require laravel/sanctum
```

Después:

1. Borrad `tests/Feature/ExampleTest.php` y `tests/Unit/ExampleTest.php` .
2. Copiad el contenido de `proyecto/` encima de la carpeta del proyecto (incluido el fichero oculto `.env.example`).
3. Cread en HeidiSQL la base de datos `academia_laravel` .
4. Y ejecutad:

terminal — php-paso-a-paso/clase38 — Visual Studio Code

terminal

```
cp .env.example .env
php artisan key:generate
php artisan migrate:fresh --seed
php artisan storage:link
php artisan test
```

En Windows, si `cp` no funciona en vuestra terminal, copiad y renombrad el fichero desde el explorador. Para tener **todo** funcionando, abrid además dos terminales más:

terminal — php-paso-a-paso/clase38 — Visual Studio Code

terminal

```
php artisan queue:work      # los correos y las notificaciones (clase 37)
php artisan schedule:work   # las tareas programadas (clase 37)
```

Los usuarios del *seeder* son `ramon@`, `marta@` y `pablo@academia-alpe.test`, con la contraseña `password` (clase 33).



Todo lo que hemos ido diciendo a lo largo del bloque, junto en una lista:

Organización

- Controladores **delgados**: reciben la petición, llaman a quien haga el trabajo y devuelven la respuesta. La validación, en *Form Requests*; los permisos, en *policies*; la lógica de negocio, en servicios o modelos.
- Una clase, una tarea. Si un método no cabe en la pantalla, probablemente hace demasiado.
- Seguid las **convenciones** de Laravel: nombres de tablas, rutas de recurso, carpetas… El que venga detrás os lo agradecerá.

Base de datos

- Siempre **migraciones**; nunca cambiéis la estructura a mano en HeidiSQL.
- Una migración ya compartida no se modifica: se crea otra.
- `with()` y `withCount()` para evitar el problema N+1 (y `preventLazyLoading()` para que os avise).
- `Fillable` en todos los modelos: nunca `create($request->all())`.

Seguridad

- Validad **todo** lo que llega del usuario, en el servidor.
- `{{ }}` en las vistas; `{!! !!}` solo con HTML vuestro.
- `@csrf` en cada formulario y `POST` para todo lo que cambie datos.
- Autorizad en el servidor (rutas, controladores, *Form Requests*), no solo ocultando botones.
- El `.env` nunca va a Git, y en producción `APP_DEBUG=false`.

Configuración

- `env()` solo dentro de `config/`; en el código, `config()`.
- Nada de valores «mágicos» repetidos (el IVA, el teléfono): a `config/academia.php`.

Calidad

- Pruebas para lo importante, y una prueba nueva por cada error que arregléis.
- Pint antes de cada *commit*.
- Commits pequeños, con mensajes que expliquen el **porqué**.



CLASE 38 · PUNTO 4

🔍 Revisión de código

En un equipo, nadie sube código a la rama principal sin que **otra persona lo revise**. Se hace con las ***pull requests*** de GitHub (o *merge requests* de GitLab): proponéis vuestros cambios en una rama, un compañero los lee, comenta y, cuando todo está bien, se fusionan. Sirve para encontrar errores, pero sobre todo para **aprender unos de otros**.

Antes de pedir una revisión: las herramientas

Lo primero es que las máquinas revisen lo que pueden revisar solas.

Pint viene instalado con Laravel y deja el código con un estilo uniforme [1]:

terminal — php-paso-a-paso/clase38 — Visual Studio Code

terminal

```
./vendor/bin/pint          # corrige el estilo de todos los ficheros
./vendor/bin/pint --test   # solo avisa, sin cambiar nada
```

Pint usa por defecto el estilo de Laravel, que tiene un par de diferencias con el que hemos usado en el curso. En `pint.json` lo adaptamos:

datos.json — php-paso-a-paso/clase38 — Visual Studio Code

datos.json

```json
{
    "preset": "laravel",
    "exclude": [
        "bootstrap",
        "config",
        "public"
    ],
    "rules": {
        "concat_space": {
            "spacing": "one"
        },
        "php_unit_method_casing": {
            "case": "camel_case"
        },
        "new_with_parentheses": {
            "named_class": true,
            "anonymous_class": false
        }
    }
}
```

- `concat_space`: un espacio a cada lado del punto al concatenar (`'Hola ' . $nombre`), como hemos escrito siempre.
- `php_unit_method_casing`: los nombres de las pruebas en *camelCase* (`unEditorPuedeCrearUnCurso`), como el resto de métodos.
- `exclude`: no tocamos los ficheros que trae Laravel.

Cuando lo pasé por la academia, cambió cosas como esta, que Laravel prefiere en una línea:

ejemplo.php — php-paso-a-paso/clase38 — Visual Studio Code

ejemplo.php

```php
// Antes
public function __construct(
    public Curso $curso,
) {
}

// Después de Pint
public function __construct(
    public Curso $curso,
) {}
```



💡 **Larastan** (`larastan/larastan`) es PHPStan (clase 21) adaptado a Laravel: encuentra errores de tipos, métodos que no existen o relaciones mal escritas **sin ejecutar** el código.

Qué mirar en una revisión

Pregunta	Ejemplo de comentario
¿Funciona? ¿Hay pruebas?	«¿Qué pasa si el curso no tiene fecha de inicio? Añadiría una prueba»
¿Es seguro?	«Este `orderBy()` usa un dato de la URL sin comprobarlo»
¿Valida y autoriza?	«Falta el `Gate::authorize()` en `destroy()`»
¿Hay consultas de más?	«Aquí hay un N+1: falta un `with('categoria')`»
¿Se entiende?	«`$d` no dice nada; ¿`$descuento`?»
¿Está en su sitio?	«Este cálculo se repite en la API: lo llevaría a un servicio»
¿Sigue las convenciones?	«La tabla debería llamarse `profesores` y el modelo `Profesor`»

Y dos normas de convivencia: se comenta **el código, no a la persona** («esta función es larga», no «haces funciones larguísimas»), y se explica **por qué** se propone un cambio.



Producción es el servidor donde la usan los clientes de verdad. Allí, la configuración cambia [2]:

ejemplo.php — php-paso-a-paso/clase38 — Visual Studio Code

ejemplo.php

```
APP_NAME="Academia Alpe"
APP_ENV=production
APP_KEY=base64:...            # una NUEVA, generada en el servidor
APP_DEBUG=false               # ¡IMPRESCINDIBLE!
APP_URL=https://academia-alpe.es

LOG_CHANNEL=daily
LOG_LEVEL=warning

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_DATABASE=academia
DB_USERNAME=academia          # un usuario propio, nunca root
DB_PASSWORD=una-contraseña-larga-y-difícil

QUEUE_CONNECTION=database
CACHE_STORE=database

MAIL_MAILER=smtp              # un servidor de correo de verdad
MAIL_HOST=smtp.proveedor.com
MAIL_USERNAME=...
MAIL_PASSWORD=...
```



⚠️ Con `APP_DEBUG=true` en producción, cualquier error enseña al mundo vuestro código, las rutas del servidor y, a veces, las contraseñas del `.env`. Es el fallo de seguridad más común en aplicaciones Laravel publicadas.

Optimizar

En producción, Laravel puede juntar la configuración, las rutas, las vistas y los eventos en ficheros de caché, para no leerlos en cada petición:

terminal — php-paso-a-paso/clase38 — Visual Studio Code

terminal

```
php artisan optimize        # config:cache + route:cache + view:cache + event:cache
php artisan optimize:clear  # deshace todo lo anterior
```

Recordad lo de la clase 27: con la configuración en caché, `env()` fuera de `config/` devuelve `null`.



CLASE 38 · PUNTO 6

🤔 Dónde publicar una aplicación Laravel

Opción	Cómo es	Para…
Hosting compartido	Barato, con cPanel. Muchos no dejan usar la terminal, ni Composer, ni colas, ni elegir la carpeta `public/`	Webs muy sencillas (y con cuidado)
Servidor virtual (VPS)	Un Linux entero para vosotros (DigitalOcean, Hetzner, OVH, Arsys…). Lo configuráis todo	Aprender cómo funciona todo y tener control total
Laravel Forge o **Ploi**	Paneles que configuran y mantienen vuestro VPS: Nginx, PHP, MySQL, colas, cron, HTTPS…	Trabajo profesional sin ser administrador de sistemas
Laravel Cloud	La plataforma oficial: conectáis el repositorio de GitHub y se encarga de todo	Despreocuparse del servidor

Antes de pagar nada, comprobad que el servidor tiene lo que pide Laravel 13 [3]: **PHP 8.3** o superior con las extensiones `ctype`, `curl`, `dom`, `fileinfo`, `filter`, `hash`, `mbstring`, `openssl`, `pcre`, `pdo`, `session`, `tokenizer` y `xml`, **Composer**, una base de datos y poder apuntar el dominio a la carpeta `public/`.



CLASE 38 · PUNTO 7

📘 Despliegue paso a paso en un servidor

Estos son los pasos en un VPS con Ubuntu, Nginx, PHP-FPM y MySQL ya instalados. Merece la pena hacerlo **al menos una vez** a mano: después, Forge o Cloud os parecerán magia comprensible. 😉

1. Bajar el código

terminal — php-paso-a-paso/clase38 — Visual Studio Code

terminal

```
cd /var/www
git clone https://github.com/vuestro-usuario/academia-laravel.git academia
cd academia
```

2. Instalar las dependencias, sin las de desarrollo

terminal — php-paso-a-paso/clase38 — Visual Studio Code

terminal

```
composer install --no-dev --optimize-autoloader
```

`--no-dev` no instala lo que solo sirve en desarrollo (PHPUnit, Pint...), y `--optimize-autoloader` genera una autocarga más rápida.

3. El `.env` de producción

terminal — php-paso-a-paso/clase38 — Visual Studio Code

terminal

```
cp .env.example .env
nano .env                    # rellenad los datos del punto 5
php artisan key:generate
```

4. La base de datos y los archivos

terminal — php-paso-a-paso/clase38 — Visual Studio Code

terminal

```
php artisan migrate --force
php artisan db:seed --class=CategoriaSeeder --force
php artisan storage:link
```

`--force` es obligatorio en producción: Laravel pide confirmación antes de tocar la base de datos de verdad. Y ojo: en producción **no** ejecutamos `migrate:fresh` ni el `DatabaseSeeder` completo, que llenaría la web de alumnos inventados. Solo los datos fijos, como las categorías.

5. Los permisos

El servidor web tiene que poder escribir en `storage/` y en `bootstrap/cache/`:

terminal — php-paso-a-paso/clase38 — Visual Studio Code

terminal

```
sudo chown -R www-data:www-data storage bootstrap/cache
sudo chmod -R 775 storage bootstrap/cache
```

6. El servidor web

Nginx tiene que apuntar a la carpeta `public/` y mandar todas las peticiones a `index.php`, igual que hacía nuestro `.htaccess` de la clase 22. Os dejo la configuración recomendada en `nginx-academia.conf`:

ejemplo.php — php-paso-a-paso/clase38 — Visual Studio Code

ejemplo.php

```
root /var/www/academia/public;

location / {
    try_files $uri $uri/ /index.php?$query_string;
}
```

7. HTTPS

Hoy ninguna web puede ir sin HTTPS (y las cookies de sesión lo necesitan). **Let's Encrypt** da certificados gratuitos:

terminal — php-paso-a-paso/clase38 — Visual Studio Code

terminal

```
sudo certbot --nginx -d academia-alpe.es -d www.academia-alpe.es
```

8. Las colas

En el servidor no hay nadie para abrir una terminal con `queue:work`. **Supervisor** es un programa que mantiene el *worker* siempre en marcha y lo relanza si se cae (`supervisor-academia-worker.conf`):

ejemplo.php — php-paso-a-paso/clase38 — Visual Studio Code

ejemplo.php

```
[program:academia-worker]
command=php /var/www/academia/artisan queue:work --sleep=3 --tries=3 --max-time=3600
autostart=true
autorestart=true
user=www-data
numprocs=2
```

terminal — php-paso-a-paso/clase38 — Visual Studio Code

terminal

```
sudo supervisorctl reread
sudo supervisorctl update
sudo supervisorctl start "academia-worker:*"
```

9. Las tareas programadas

Una sola línea en el `cron` del sistema (`crontab.txt`): cada minuto, Laravel mira si toca alguna tarea de las que programamos en la clase 37.

Esquema

```
* * * * * cd /var/www/academia && php artisan schedule:run >> /dev/null 2>&1
```

10. Optimizar

terminal — php-paso-a-paso/clase38 — Visual Studio Code

terminal

```
php artisan optimize
```

¡Y a probar en el navegador! 🎉



CLASE 38 · PUNTO 8

😮 Actualizar sin romper nada

Cada vez que haya cambios, se repite una parte de lo anterior. Para no olvidar ningún paso, lo guardamos en un script, `deploy.sh`:

terminal — php-paso-a-paso/clase38 — Visual Studio Code

terminal

```
#!/bin/bash
# Actualiza la academia en el servidor: bash deploy.sh
set -e

php artisan down --retry=60                          # modo mantenimiento
git pull origin main                                 # el código nuevo
composer install --no-dev --optimize-autoloader --no-interaction
php artisan migrate --force                          # las migraciones nuevas
php artisan optimize                                 # rehacer las cachés
php artisan queue:restart                            # los workers cargan el código nuevo
php artisan up                                       # ¡de vuelta!
```

- `set -e` detiene el script en cuanto un paso falle: mejor parar que seguir a medias.
- `php artisan down` pone la web en **modo mantenimiento**: los visitantes ven un aviso (un error 503) mientras actualizamos. Con `--secret=una-clave` podéis entrar vosotros por `https://academia-alpe.es/una-clave` para comprobar que todo va bien antes del `up`.
- `queue:restart`: ¿recordáis que el *worker* carga el código al arrancar (clase 37)? Esto le dice que termine lo que está haciendo y vuelva a empezar con la versión nueva.



💡 Forge, Ploi y Laravel Cloud hacen este proceso con **cero segundos de corte** (*zero-downtime deployment*): preparan la versión nueva en otra carpeta y, cuando está lista, cambian un enlace simbólico. El visitante ni se entera.



CLASE 38 · PUNTO 9

🚀 El proyecto final

Ahora os toca a vosotros. El proyecto final es una aplicación Laravel **vuestra**, de principio a fin. Se puede hacer de forma individual o en grupos de dos o tres personas.

Propuestas

Proyecto	Idea
📚 **Biblioteca**	Libros, autores y socios; préstamos con fecha de devolución y aviso por correo de los retrasados
🏋️ **Gimnasio**	Clases, monitores y reservas con plazas limitadas; los socios ven sus reservas
🍽️ **Restaurante**	Carta por categorías, reservas de mesa y panel para el personal
🏠 **Inmobiliaria**	Pisos con fotos, filtros por precio y zona, y formulario de contacto por anuncio
🎫 **Incidencias**	Usuarios que abren incidencias, técnicos que las resuelven, estados y comentarios
💡 **La vuestra**	Una idea propia, con mi visto bueno antes de empezar

Requisitos mínimos

- ☐ Al menos **cuatro tablas** con migraciones, con una relación uno a muchos y otra muchos a muchos.
- ☐ *Seeders* y *factories* para tener datos de prueba con un solo comando.
- ☐ Un **CRUD completo** con validación en un *Form Request* y mensajes en español.
- ☐ Subida de al menos un tipo de archivo (imagen o documento).
- ☐ **Login** y al menos **dos roles** con permisos distintos (*policy* o *gates*).
- ☐ Un *layout* Blade con componentes y paginación en los listados.
- ☐ Una **API** con al menos un recurso de lectura (*API Resource*).
- ☐ Al menos **diez pruebas** automáticas que pasen.
- ☐ Un **evento** con su *listener*, o un correo enviado desde la cola.
- ☐ El código pasado por **Pint**, en un repositorio de **Git** con *commits* frecuentes y con sentido.
- ☐ Un `README.md` que explique qué hace y cómo instalarla.
- ☐ ⭐ Para nota: publicada en Internet.

Cómo lo valoraré

Aspecto	Qué miro
Funciona	Se instala siguiendo el `README` y hace lo que promete
Código	Organizado, legible, con los nombres y las convenciones del curso
Seguridad	Valida, autoriza y no tiene los fallos que hemos ido viendo
Pruebas	Prueban lo importante, y pasan
Git	La historia de *commits* cuenta cómo se construyó el proyecto
Presentación	Una demo de diez minutos explicando qué habéis hecho y qué habéis aprendido



💡 **Mi consejo:** empezad por las **migraciones** y los **modelos** (dibujad antes las tablas en un papel), después el CRUD más importante, y dejad para el final lo «bonito». Una aplicación pequeña que funciona bien vale mucho más que una enorme a medio hacer.



CLASE 38 · PUNTO 10

🚀 Ejercicios

Las soluciones se copian dentro del proyecto, respetando sus carpetas.

- ☐ **01 · Página de mantenimiento.** Cread `resources/views/errors/503.blade.php` con un aviso amable y el teléfono de la academia, y probadla con `php artisan down`. Cuidado: **no** uséis el layout, porque durante el mantenimiento puede que la base de datos no esté disponible (y el menú consulta las categorías). → `errors/503.blade.php`
- ☐ **02 · Lista de comprobación.** Cread con `php artisan make:command ComprobarDespliegue` un comando **de clase** (en `app/Console/Commands`) llamado `academia:comprobar` que revise que la aplicación está lista para producción: `APP_KEY` generada, `APP_DEBUG` desactivado, entorno `production`, `https` en `APP_URL`, conexión con la base de datos, enlace `public/storage`, permisos de `storage/`, cola distinta de `sync` y correo distinto de `log`. Que termine con error si algo falla. → `ComprobarDespliegue.php`
- ☐ **03 · Integración continua.** Cread un *workflow* de **GitHub Actions** que, en cada `push` y en cada *pull request*, instale el proyecto, compruebe el estilo con Pint y ejecute las pruebas. Así nadie puede fusionar código que rompa algo. → `.github/workflows/pruebas.yml`



🏆 **Reto final:** publicad la academia (o vuestro proyecto) en Internet. Un VPS de los más pequeños cuesta unos pocos euros al mes, y Laravel Cloud tiene opciones para empezar. Mandadme el enlace: ¡quiero verla! 🚀



CLASE 38 · PUNTO 11

👋 Lo que tenéis que llevaros del curso

Idea	En una frase
Programar	Pensar el problema antes de escribir el código
PHP y POO	Clases pequeñas, con una tarea, que colaboran entre sí
Nuestro MVC	Ya sabéis qué hay debajo de cualquier *framework*
Laravel	Las mismas piezas, probadas por millones de proyectos
Seguridad	Validar, escapar, autorizar y no confiar nunca en lo que llega del usuario
Calidad	Pruebas, Pint y revisiones de código
Producción	`APP_DEBUG=false`, HTTPS, colas con Supervisor y una línea de `cron`
Seguir aprendiendo	La documentación oficial es vuestra mejor amiga

🎓 Despedida

Si habéis llegado hasta aquí, habéis hecho algo que mucha gente empieza y pocos terminan. Recordad cómo os sentíais la primera vez que visteis un `<?php` en la clase 4, y mirad ahora vuestra academia: rutas, base de datos, usuarios, API, pruebas, colas…

No os voy a engañar: esto no se acaba aquí. En informática **nunca** se termina de aprender, y eso es precisamente lo bonito. Seguid haciendo proyectos, leed código de otros, equivocaos mucho (es la única forma de aprender), y cuando algo no salga, respirad hondo, leed el mensaje de error con calma… y a por ello. 💪

Ha sido un placer acompañaros. ¡Nos vemos en el código!

Ramón

📚 Fuentes y documentación oficial

1. Laravel: Pint
2. Laravel: Configuration, Environment Configuration
3. Laravel: Deployment
4. Laravel: Queues, Supervisor Configuration
5. Laravel: Task Scheduling, Running the Scheduler
6. Laravel: Configuration, Maintenance Mode
7. Let's Encrypt: Certbot
8. GitHub Docs: GitHub Actions